



**MAT-350: Project 2— Application of Lower-Rank
Approximation to Image Compression**

Prepared on: August 17, 2026
Prepared for: Prof. K. B. Gardner
Prepared by: Alexander Ahmann

Contents

1	Background	2
1.1	Problem Setup	3
2	Worked Solutions	3
2.1	Part I— Q1: Rank-1 approximation on arbitrary matrix A . .	3
2.2	Q2: Rank-2 approximation on A	5
2.3	Q3: Dot Products and Cross Products of U_A vectors	7
2.4	Q4: Determining whether-or-not U_A spans $\mathbb{R}^3$	8
2.5	Part II— Q5: Applying the lower-rank approximation compression technique to <code>image.mat</code> for the case that $CR \approx 2$. .	9
2.6	Q6: Resulting Compressed Image, and its RMSE	11
2.7	Q7: The low-rank approximation compression technique for cases $CR \approx 10$, $CR \approx 25$, and $CR \approx 75$	12
3	Discussion	14
3.1	Assessment of low-rank compression results	14
3.2	Limitations: Caveat Lector	15
3.3	Limitations: Experimental Design	16
4	Conclusion	17
4.1	Acknowledgements	17
A	Resulting images after the lower-rank approximation compression technique	18

B	A (very short) Introduction to Linear Algebra	24
B.1	Matrix Algebra	24
B.2	Vector Algebra	26
C	MATLAB Solution Source Code	28
D	Results of Running the MATLAB solution	32

1 Background

In this project, I assume the role of a computer programmer for a popular social media site.¹ A feature of the site is the storage of image files uploaded by a large number of users. To economize and avoid having to overspend for web storage, I will use a technique of k -rank approximations for compression, which I will demonstrate later in this paper,² but a high-level overview is that k -rank approximation for compression makes use of *Singular Value Decomposition*, which in this context involves expressing images in terms of a matrix, factoring the matrix into three components such that,

$$A = U\Sigma V^T \quad (1)$$

with A being the image matrix. Then, the lower-rank approximation involves selecting a subset of the U , Σ and V^T matrices such that their columns, and rows in the case of the Σ , are between 1 and some rank- k value in accordance with a target compression ratio, which is defined as,

$$CR = \frac{mn}{k(m+n+1)} \quad (2)$$

where m is the number of rows in the image matrix and n is the number of columns in the image matrix. The compressed image in terms of its k -rank worked out as:

$$A_k = U_{:,k}\Sigma_{k,k}V_{:,k}^T \quad (3)$$

and then the image matrix is encoded back to an image. As the k increases, this method will distort the image, and this project is a proof-of-concept showing how an increase in the k in the low-rank approximation further distorts the resemblance between the original matrix and the k -rank approximation matrix, and in the case of image compression, distorts the original image.

¹After the MAT-350 (n.d.-a, Scenario).

²MAT-350 (n.d.-b) elaborates further in the technique for applying SVD to image compression.

1.1 Problem Setup

This project consists of two parts:

- **Part I** consists of a MATLAB solution to a toy-problem consisting of an arbitrary matrix A , and applying the method of singular-value decomposition to the matrix, and then applying low-rank approximations to assess how closely the lower-rank resembles the original matrix given the k^{th} rank of the matrix. Such resemblance is worked out through the *root mean squared error* formula. The metrics produced by the toy problem is to then be used as a basis of comparison for the task at hand, which as stated is to compress images.
- **Part II** is the realistic problem in where an image is loaded into MATLAB, and then the singular-value decomposition and low-rank approximation is applied to the image. Different values for k low ranks are proposed, and a “compression ratio” formula is used to derive different k -values for a low-rank approximation, given a desired compression ratio.

2 Worked Solutions

2.1 Part I— Q1: Rank-1 approximation on arbitrary matrix A

“Consider the matrix 3×3 :

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 3 & 4 \\ 5 & 6 & 7 \end{bmatrix}$$

Use the `svd()` function in MATLAB to compute A_1 , the **rank-1 approximation of A** . Clearly state what A_1 is, rounded to 4 decimal places. Also, **compute** the root mean square error (RMSE) between A and A_1 .”—MAT-350 (n.d.-a, Step 1)

The first four (4) questions concerns themselves with the application of the linear algebra needed to perform image compression on a sample matrix. In particular, the built in MATLAB function `svd()` to work out the rank-1 and rank-2 *low-rank approximations*, and then apply the *root mean squared error* formula to work out how well that a particular resulting lower-rank approximation, be it rank-1, rank-2, or some higher k -value for rank, resembles

the original matrix.

My solution to this problem is as given,

```
A = [1 2 3;
      3 3 4;
      5 6 7]

[U, S, V] = svd(A)

A_1 = U(:, 1:1) * S(1:1, 1:1) * V(:, 1:1)'
A_norm1 = norm(A - A_1, 'fro') / 3
A_rmse1 = sqrt(mean((A - A_1).^2, 'all'))
A_rank1 = rank(A_1)
```

Running this code results in,

A =

1	2	3
3	3	4
5	6	7

U =

-0.2904	0.9504	-0.1114
-0.4644	-0.2418	-0.8520
-0.8367	-0.1957	0.5115

S =

12.5318	0	0
0	0.9122	0
0	0	0.3499

V =

-0.4682	-0.8261	-0.3136
---------	---------	---------

```
-0.5581    0.0012    0.8298
-0.6851    0.5635   -0.4616
```

```
A_1 =
```

```
1.7039    2.0313    2.4935
2.7243    3.2477    3.9867
4.9087    5.8517    7.1832
```

```
A_norm1 =
```

```
0.3257
```

```
A_rmse1 =
```

```
0.3257
```

```
A_rank1 =
```

```
1
```

The script applies the SVD and decomposes the **A** into three separate matrices, **U**, **S**, and **V**, such that the product **USV'** will result in the original matrix **A**. Next, the rank-1 approximation of the matrix is computed, and both the Frobenius norm and root mean square errors for this rank-1 approximation are worked, and stored in **A_norm1** and **A_rmse1**, respectively. Finally, the rank of the **A_1** matrix is worked, stored in **A_rank1**, and I can confidently conclude that this is a rank-1 approximation of the given matrix, and that its RMSE is 0.3257.

2.2 Q2: Rank-2 approximation on *A*

“Use the **svd()** function in MATLAB to computer A_2 , the **rank-2 approximation of A** . Clearly state what A_2 is, rounded to 4 decimal places. Also, **compute** the root mean square error (RMSE) between A and A_2 . Which approximation is better, A_1 or A_2 ? Explain.”—MAT-350 (n.d.-a, Step 2)

This problem is just a repeat of the previous problem above, but with the slight change to compute a rank-2 approximation. My solution to this problem is as given,

```
A_2 = A_1 + U(:, 1:2) * S(1:2, 1:2) * V(:, 1:2).'
```

```
A_norm2 = norm(A - A_2, 'fro') / 3
```

```
A_rmse2 = sqrt(mean((A - A_2).^2, 'all'))
```

```
A_rank2 = rank(A_2)
```

Running this code results in,

```
A_2 =
```

```
    2.6917    4.0636    5.4755
    5.6308    6.4950    7.8490
    9.9648   11.7032   14.2659
```

```
A_norm2 =
```

```
    4.1789
```

```
A_rmse2 =
```

```
    4.1789
```

```
A_rank2 =
```

```
    2
```

The relevant metrics here are: $A_rmse2 = 4.1789$, and $A_rank2 = 2$. The latter confirms that the procedure correctly produced a rank-2 approximation, and that the RMSE calculation for the rank-2 matrix is 4.1789. From these results, the RMSE for $A_1 = 0.3257$ and the RMSE for $A_2 = 4.1789$, I can conclude that the A_1 is a better approximation than the A_2 , because the RMSE measures the *squared error*.

This means that the resulting RMSE coefficient increases as the rank- k approximation gets “further away” from resembling the original matrix— or a larger error. I posit that the rank- k approximation which better resembles

the given matrix is the superior approximation, and therefore have concluded that a rank-1 approximation expressed in the resulting A_1 matrix is the superior lower-rank approximation.

2.3 Q3: Dot Products and Cross Products of U_A vectors

“For the 3×3 matrix A , the singular value decomposition is $A = USV'$ where $U = [\vec{u}_1, \vec{u}_2, \vec{u}_3]$. Use MATLAB to **compute** the dot product $d_1 = \text{dot}(\vec{u}_1, \vec{u}_2)$. Also, use MATLAB to **compute** the cross product $\vec{c} = \text{cross}(\vec{u}_1, \vec{u}_2)$ and dot product $d_2 = \text{dot}(\vec{c}, \vec{u}_3)$.”—MAT-350 (n.d.-a, Step 3)

This problem is just to explore the consequences of applying the dot product and the cross product to different vectors that constitute the U matrix. Note that,

$$U = [\vec{u}_1 \ \vec{u}_2 \ \vec{u}_3] \quad (4)$$

My solution to this problem is as given,

```
u_1 = U(:,1)
u_2 = U(:,2)
u_3 = U(:,3)

d_1 = dot(u_1, u_2)
c = cross(u_1, u_2)
d_2 = dot(c, u_3)
```

Running this code results in,

```
u_1 =

    -0.2904
    -0.4644
    -0.8367
```

```
u_2 =

    0.9504
   -0.2418
   -0.1957
```

u_3 =

```
-0.1114  
-0.8520  
0.5115
```

d_1 =

```
1.6653e-16
```

c =

```
-0.1114  
-0.8520  
0.5115
```

d_2 =

```
1.0000
```

An interesting observation is that the dot product of the cross product of two vectors, and the third vector, results in the same third vector. I am not sure if this is some idempotent property, and further investigation is warranted.

2.4 Q4: Determining whether-or-not U_A spans $\mathbb{R}^3$

“Using the matrix $U = [\vec{u}_1, \vec{u}_2, \vec{u}_3]$, **determine whether or not the columns of U span $\mathbb{R}^3$. Explain your approach.**”—
MAT-350 (n.d.-a, Step 4)

To work out whether-or-not a given matrix spans $\mathbb{R}^n$, the procedure is to take the *reduced row echelon form* of said matrix, and determine if the count of pivot columns is $\leq$ the n of the $\mathbb{R}^n$ space. The following code solves this problem,

```
U_rref = rref(U)
```

Running this code results in,

```
U_rref =
```

```
1    0    0
0    1    0
0    0    1
```

Because there are three pivot columns, I can conclude that the given U matrix spans $\mathbb{R}^3$.

2.5 Part II— Q5: Applying the lower-rank approximation compression technique to image.mat for the case that $CR \approx 2$

“Use the MATLAB `imshow()` function to load and display the image A stored in the provided MATLAB image.mat file (available in the Supporting Materials area). For the loaded image, **derive the value of k** that will result in a compression ratio of $CR \approx 2$. For this value of k , **construct the rank- k approximation of the image.**”—MAT-350 (n.d.-a, Step 5)

This problem is the transition from toy examples to a concrete, “real world” problem that is faced by many computer scientists in industry: “when dealing with limited time and space, how can they reduce the amount of time taken to solve a problem, and the space needed to store information?” Of course, it would be nice to reduce both the time taken and space needed whilst retaining the original information, but typically we can only decrease the space needed to store information at the cost of time taken to compute said information, decrease the time taken to compute said information at the cost of increasing the space needed to store said information, or to reduce the amount of information in a file, which results in a decrease in both space needed and time taken, but at the cost of resolution.

This technique of an low-rank image compressor takes the third approach of reducing both space needed and time taken, but at the cost of the resolution of the original file. In particular, the method involves expressing an image file in terms of a matrix, applying a SVD decomposition of the image into its respective U , Σ , and V matrices, and then applying a low-rank approximation to the matrix in order to reduce the information needed to store the file, without making a change to the time taken to compute

the information in the compressed file when rendering it, but at the cost of resolution.

A formal treatment of this problem, as previously stated in the “Background” section, is to find a U , Σ , and V matrix such that,

$$A = U\Sigma V^T \quad (5)$$

And then to select a k -value that is calibrated after the desired compression ratio, which I shall elaborate on further, and then finally work out the k low-rank of the matrix A_k such that,

$$A_k = U_{:,k}\Sigma_{k,k}V_{:,k}^T \quad (6)$$

My solution to this problem is as follows,

```
load image.mat

imshow(A)
title("Given Image (No Compression)");

[m, n] = size(A)

image2matrix = im2double(double(A));
[U S V] = svd(image2matrix);

CR = 2
k_2 = round( (m * n) / (CR * (m + n + 1)))
Ak_CR2 = U(:, 1:k_2) * S(1:k_2, 1:k_2) * V(:, 1:k_2)';

Ak_CR2_rank = rank(Ak_CR2)
Ak_CR2 = uint8(round(Ak_CR2));
```

Running this code results in,

m =

2583

n =

4220

CR =

2

k_2 =

801

Ak_CR2_rank =

801

The `m` and `n` variables represent the column-by-row dimensions of the image matrix A , as worked by the `size(A)` function, and the actual image with no compression applied to it is shown.³ The compression ratio `CR` = 2, and the k -value for the low-rank approximation is estimated in accordance with the following formula:

$$CR = \frac{mn}{k(m+n+1)} \quad (7)$$

The compression ratio is given, it should be ≈ 2 . So, this compression ratio formula should be expressed in terms of k . This is done with basic algebra, solving for k results in:

$$k = \text{round} \left(\frac{mn}{CR(m+n+1)} \right) \quad (8)$$

In the specific case of $CR \approx 2$, the k for the low-rank approximation is worked out to $k = 801$. Furthermore, the `rank()` function is invoked to cross-confirm that the k value such that $CR \approx 2$ is correct, and stored in the variable `Ak_CR2_rank`. The worked k is equal to the `Ak_CR2_rank` results, confirming that the procedure was executed correctly.

2.6 Q6: Resulting Compressed Image, and its RMSE

“Display the image and compute the root mean square error (RMSE) between the approximation and the original image.

³See figure 1.

Make sure to include a copy of the approximate image in your report.”—MAT-350 (n.d.-a, Step 6)

This part involves working the root mean squared error to determine how much the low-rank approximation resembles the original matrix, and to also display the compressed image. My solution to this problem is as given,

```
figure;
imshow(Ak_CR2);
title("Given Image (CR  $\approx$  2)");

rmse_CR2 = sqrt(mean((A - Ak_CR2).^2, 'all'))
```

Running this code results in,

```
rmse_CR2 =

    2.2389
```

In appendix A, specifically figure 2, an image is displayed showing the results of the compression. The next solution will address this approach to producing several compressed images with this technique, for varying compression ratios and their respective k -value for the low-rank approximation compression method.

2.7 Q7: The low-rank approximation compression technique for cases $CR \approx 10$, $CR \approx 25$, and $CR \approx 75$

“Repeat steps 5 and 6 for $CR \approx 10$, $CR \approx 25$, and $CR \approx 75$. **Explain** what trends you observe in the image approximation as CR increases and provide your recommendation for the best CR based on your observations. Make sure to include a copy of the approximate images in your report.”—MAT-350 (n.d.-a, Step 7)

My solution to this problem is as given,

```
%%
% Case CR  $\approx$  10
%%

CR = 10
k_10 = round( (m * n) / (CR * (m + n + 1)))
```

```

    % Derived by solving for "k" in:
    % CR = (m * n) / (k * (m + n + 1))
    Ak_CR10 = U(:, 1:k_10) * S(1:k_10, 1:k_10) * V(:, 1:k_10)';

    Ak_CR10_rank = rank(Ak_CR10)
    Ak_CR10 = uint8(round(Ak_CR10));

    figure;
    imshow(Ak_CR10);
    title("Given Image (CR ~= 10)");

    rmse_CR10 = sqrt(mean((A - Ak_CR10).^2, 'all'))

    %%
    % Case CR ~= 25
    %%

    CR = 25
    k_25 = round( (m * n) / (CR * (m + n + 1)))
    % Derived by solving for "k" in:
    % CR = (m * n) / (k * (m + n + 1))
    Ak_CR25 = U(:, 1:k_25) * S(1:k_25, 1:k_25) * V(:, 1:k_25)';

    Ak_CR25_rank = rank(Ak_CR25)
    Ak_CR25 = uint8(round(Ak_CR25));

    figure;
    imshow(Ak_CR25);
    title("Given Image (CR ~= 25)");

    rmse_CR25 = sqrt(mean((A - Ak_CR25).^2, 'all'))

    %%
    % Case CR ~= 75
    %%

    CR = 75
    k_75 = round( (m * n) / (CR * (m + n + 1)))
    % Derived by solving for "k" in:
    % CR = (m * n) / (k * (m + n + 1))
    Ak_CR75 = U(:, 1:k_75) * S(1:k_75, 1:k_75) * V(:, 1:k_75)';

```

```

Ak_CR75_rank = rank(Ak_CR75)
Ak_CR75 = uint8(round(Ak_CR75));

figure;
imshow(Ak_CR75);
title("Given Image (CR  $\approx$  75)");

rmse_CR75 = sqrt(mean((A - Ak_CR75).^2, 'all'))

```

Rather than just posting the raw MATLAB results in this subsection, I have decided to format them into table 1 to more easily illustrate their differences,⁴

Compression Ratio	k -value	RMSE	Figure	File Size
No Compression	N/A	N/A	Fig. 1	781 [kb]
$CR \approx 2$	801	2.2389	Fig. 2	781 [kb]
$CR \approx 10$	160	5.1769	Fig. 3	747 [kb]
$CR \approx 25$	64	6.3933	Fig. 4	621 [kb]
$CR \approx 75$	21	7.6909	Fig. 5	513 [kb]

Table 1: Results of Image Compression

Again, the resulting compressed images can be viewed at appendix A.

3 Discussion

3.1 Assessment of low-rank compression results

From table 1, I have noticed that quite a few interesting patterns emerge between comparisons of the different features:

- There is an negative relationship between compression ratio and k -value for low-rank approximation, that is: as the CR increases, the k decreases.
- There is an positive relationship between compression ratio and root mean square error, that is, as the CR increases, so too does the the $RMSE$ increase.

⁴The raw MATLAB output is in the appendices of this document.

- Unsurprisingly, as the compression ratio increases, the resulting file size decreases.
- As the k -value for low-rank approximation decreases, the root mean square error increases.

Some of these relationships are consistent with the ones discovered when working with the toy problem. Chief amongst these relationship is the negative correlation between the k -value for a low-rank approximation, and the root mean squared error value. As the k increases, the root mean squared error decreases. Specific to the problem of image compression, as the compression ratio increases, both the root mean square values (RMSE) increase, and the file size of the resulting image decreases.

However, there is a cost between increased compression ratio, and information loss: compare figure 3 with a compression ratio of approximately 10, with figure 5 with a compression ratio of approximately 75. The $CR \approx 10$ image is only “slightly distorted,” whereas the $CR \approx 75$ is abundantly distorted and barely recognisable, although the “viewability” is a bit subjective, or “in the eye of the beholder.”

3.2 Limitations: Caveat Lector

Unfortunately, due to time constraints and other reasons,⁵ I could not further scrutinise the results and implementation of my solution. The execution of the experiment, as well as the analysis of the results, may indeed be fallacious.

To give an example, I use the following bit of code to perform an SVD operation on the toy matrix in the first part involving the toy matrix problem,

```
[U, S, V] = svd(A)
```

However, I used the following bit of code to perform the very same SVD operation on the image matrix,

```
[U S V] = svd(image2matrix);
```

Notice the difference between the `[U, S, V]` and `[U S V]` in the variants of storing the results of the `svd(Matrix)` operation— there is an inconsistency in the coding style,⁶ which is bad in terms of proper software

⁵Such as personal life affairs, the “learning curve” for the MATLAB language, and admittedly sloppy execution of this experiment.

⁶Commas (,) are used in the toy example, but no commas are used in the image compression proof-of-concept.

development. Another example is that I use terms like “lower-rank approximation,” “low-rank approximation,” “low-rank compression method,” and “low-rank approximation compression procedure” interchangeably, without defining them more rigourously or at the very least clarifying that they are synonymous.

These are just a couple of examples of flaws in this project that I managed to catch while proofreading this paper, and it is very likely that there are more flaws that I have not yet identified. Further development of this proof-of-concept is needed, and a refactoring of the source code is needed to ensure that it is in accordance with modern coding standards and efficient, bug-free code.

3.3 Limitations: Experimental Design

It should be noted that the cost-benefit analysis between image distortion and space saving is a bit subjective. One person may struggle to view the $CR \approx 75$ image, whereas another person with “superior eye sight” or a “keen mind for imagination” may be able to reconstruct that very same $CR \approx 75$ image with “near-perfect” accuracy, or imagine the original uncompressed image just from the $CR \approx 75$ variant.

Computer science may be more “objective,” but the human mind is more complicated and subjective. Indeed, the subset of computer science that they call “human-computer interaction” is tasked with designing systems that can be used by all kinds of different people, regardless of their technical background,⁷ or any potential neurodivergent traits and/or learning disabilities.

If the goal is to simply save on computer storage space without regard to the resolution of the image or graphic, then one would simply make the CR variable in the k -value formula as large as possible without having it result in an undefined fraction, i.e. when $CR(m + n + 1) = 0$, or resulting in a k value that is < 0 . But, if the goal is to compress the image to save on storage space *without* distorting the image beyond recognition for users, then more sophisticated experiments are called for. For one thing, this experiment needs to be repeated on a variety of different images that were developed with different methods.⁸ Then, the lower-rank approximate compression technique needs to be applied at a wider spectrum of values for CR and its resulting k -value, and to a more diverse set of images and graphics. Next, “beta testers” are needed to determine how much image distortion that they can tolerate,

⁷Amongst other tasks, Human-Computer Interaction is more complicated than how I just described it.

⁸Like images taken from a smartphone’s camera versus images created in *Adobe Flash*.

and to show them images and graphics with a given CR ratio and ask if they can tolerate the distortion.

From this experiment and analysis alone, I cannot recommend an “optimal” CR value that would balance both space saved and “not too much distortion” of the original image. I can only say that in my subjective opinion, the $CR \approx 10$ image is the least distorted and most storage space saving image. Further research needs to be done.

4 Conclusion

The following are basic facts that I was able to work out from investigation:

- For low-rank approximations, as the k -value for the low rank increases, the root mean squared error value increases. The key consequence is that when using a rank matrix of a higher k value will result in further distortions between said k matrix and the original matrix.
- Regarding the worked k -value given a desired CR , the file size of the image decreases as k increases, but the root mean squared error of the resulting image matrix increases, and the rendered image becomes more, and more distorted as the CR increases.

Now, regarding my personal opinion on how to proceed with deciding on a compression ratio that saves storage space without distorting the resulting compressed image too much, I recommend a $CR \approx 10$. But further research will need to be done to determine how much image distortion, as a result of the low-rank approximation technique, will be tolerated by end-users before deploying this technique into production.

4.1 Acknowledgements

“Karl,” for their implementation of computing low-rank approximations,⁹ John D’Errico for their solution to a root-mean squared error formula,¹⁰ “Professor Dave Explains” in general for their linear algebra tutorial videos,¹¹ and finally Professor K. B. Gardner for her guidance and advice for processing images for this project.¹²

⁹On a MATLAB Help Center thread started by “Jessica” (2024).

¹⁰As I have learnt, older versions of the MATLAB language do not have a built in `rmse` function; see the MATLAB Help Center thread started by “Joe” (2024).

¹¹See https://www.youtube.com/playlist?list=PLybg94GvOJ9En46TNCXL2n6SiqRc_iMB8

¹²See Gardner (2026).

References

Gardner, K. B. (Aug. 12, 2026). *Project Two Tips*. Southern New Hampshire University.

“Jessica” (Feb. 20, 2024). *How to do a rank-1 approximation?*, MATLAB Help Center. Retrieved on Aug. 13, 2026 from:
<https://www.mathworks.com/matlabcentral/answers/2084643-how-to-do-a-rank-1-approximation>

“Joe” (Mar. 27, 2011). *RMSE — Root mean square Error*. MATLAB Help Center. Retrieved on Aug. 13, 2023 from:
<https://www.mathworks.com/matlabcentral/answers/4064-rmse-root-mean-square-error>

MAT-350 (n.d.-a). *Project Two Guidelines and Rubric*. Southern New Hampshire University.

MAT-350 (n.d.-b). *SVD and Image Compression Overview*. Southern New Hampshire University.

zyBooks (n.d.). *MAT-350: Linear Algebra [Jul.–Aug. 2026 Edition]*.

A Resulting images after the lower-rank approximation compression technique

The following graphics were produced after processing `image.mat`, and then applying k -rank approximation such that k is derived after a compression ratio formula.



Figure 1: Resulting Image (No Compression)



Figure 2: Resulting Image ($CR \approx 2$)



Figure 3: Resulting Image ($CR \approx 10$)



Figure 4: Resulting Image ($CR \approx 25$)



Figure 5: Resulting Image ($CR \approx 75$)

B A (very short) Introduction to Linear Algebra

This section introduces the reader to basic concepts that I have learnt from an *Applied Linear Algebra* course taught by *Southern New Hampshire University* faculty.¹³ This is not meant to be an authoritative, nor a comprehensive, introduction. Instead, this section list definitions, rules and formulæ in regards to basic linear algebra. How concepts relate to each other, or build off of each other, are also briefly discussed. Much of this introduction is heavily derived from zyBooks (n.d.), and this does not represent a rigorous nor original introduction.

The solution to the problem in this project, and to the problem in the previous project, draw from these basic principles. A discussion of these ideas will hopefully give readers additional context to better understand the problems, and possibly to “get them in a rabbit hole” for learning the skills needed to reproduce the solutions as described in this paper.¹⁴

B.1 Matrix Algebra

1. A *vector* is a mathematical object with magnitude and direction, represented by an arrow (zyBooks, n.d., §3.1). Notation used to define a vector can either be in boldface (e.g. $\mathbf{v}$) or with an arrow (e.g. $\vec{v}$).¹⁵
2. A formal treatment of the vector is: the set of all n -tuples, denoted as $\mathbb{R}^n$, which can be seen as the set of vectors, $[\vec{v}_1, \dots, \vec{v}_n]$, or the set of coordinates, $(v_1, \dots, v_n)$.
3. A *matrix* is a row-by-columns grid of real numbers,¹⁶ typically with m being the number of rows, and n being the number of columns.
4. For some row i and column j , the *minor* $M_{i,j}$ of a matrix A is a version of said matrix where the i^{th} row is deleted and the j^{th} column is deleted.

¹³As a part of their undergraduate computer science programme: MAT-350 Jul.–Aug., 2026 semester.

¹⁴This section exists in part for the author’s benefit: as something that they can reference when needing to work with applied linear algebra in real life.

¹⁵I personally prefer the latter $\vec{v}$ notation, since writing down “boldface” on paper is difficult.

¹⁶Actually, the cells in the grid can store any kind of mathematical symbol, but I shall work specifically with real numbers in the cells to keep it simple and in the context of this course.

5. The *cofactor* of a matrix is defined as: $C_{i,j} = (-1)^{i+j} \times M_{i,j}$ for row i and column j .

6. A *lower triangular matrix* is a kind of matrix in the form of:

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & a_{1,4} \\ a_{2,1} & a_{2,2} & a_{2,3} & a_{2,4} \\ a_{3,1} & a_{3,2} & a_{3,3} & a_{3,4} \\ a_{4,1} & a_{4,2} & a_{4,3} & a_{4,4} \end{bmatrix} \longrightarrow \begin{bmatrix} a_{1,1} & 0 & 0 & 0 \\ a_{2,1} & a_{2,2} & 0 & 0 \\ a_{3,1} & a_{3,2} & a_{3,3} & 0 \\ a_{4,1} & a_{4,2} & a_{4,3} & a_{4,4} \end{bmatrix} \quad (9)$$

7. An *upper triangular matrix* is a kind of matrix in the form:

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & a_{1,4} \\ a_{2,1} & a_{2,2} & a_{2,3} & a_{2,4} \\ a_{3,1} & a_{3,2} & a_{3,3} & a_{3,4} \\ a_{4,1} & a_{4,2} & a_{4,3} & a_{4,4} \end{bmatrix} \longrightarrow \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & a_{1,4} \\ 0 & a_{2,2} & a_{2,3} & a_{2,4} \\ 0 & 0 & a_{3,3} & a_{3,4} \\ 0 & 0 & 0 & a_{4,4} \end{bmatrix} \quad (10)$$

8. The *lower-upper decomposition*, or LU-decomposition, refers to a matrix factoring procedure that, given some arbitrary matrix A , it can be decomposed into a pair of matrices L and U such that:

$$A = L \times U$$

$$\begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & a_{1,4} \\ a_{2,1} & a_{2,2} & a_{2,3} & a_{2,4} \\ a_{3,1} & a_{3,2} & a_{3,3} & a_{3,4} \\ a_{4,1} & a_{4,2} & a_{4,3} & a_{4,4} \end{bmatrix} = \begin{bmatrix} l_{1,1} & 0 & 0 & 0 \\ l_{2,1} & l_{2,2} & 0 & 0 \\ l_{3,1} & l_{3,2} & l_{3,3} & 0 \\ l_{4,1} & l_{4,2} & l_{4,3} & l_{4,4} \end{bmatrix} \times \begin{bmatrix} u_{1,1} & u_{1,2} & u_{1,3} & u_{1,4} \\ 0 & u_{2,2} & u_{2,3} & u_{2,4} \\ 0 & 0 & u_{3,3} & u_{3,4} \\ 0 & 0 & 0 & u_{4,4} \end{bmatrix}$$

9. The *span* of a matrix is a set of elements $S = \{\vec{v}_1, \vec{v}_2, \dots, \vec{v}_n\}$ of some arbitrary vector space V that contains all of its linear combinations; formally put:

$$\text{Span}(\vec{v}_1, \vec{v}_2, \dots, \vec{v}_n) = \left\{ \sum_{i=1}^n a_i \vec{v}_i \mid a_i \in \mathbb{R}, 1 \leq i \leq n \right\} \quad (11)$$

From this basic notion, vectors in the form of

$$\vec{a} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} ; \quad \vec{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} ; \quad \vec{c} = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix}$$

Can be written like

$$M = \begin{bmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ \vdots & \vdots & \vdots \\ a_n & b_n & c_n \end{bmatrix}$$

B.2 Vector Algebra

1. The zero-vector $\vec{0}$ is a special case of a vector where all its components are 0.
2. The *magnitude* of a vector, denoted as $||\vec{v}||$, is the length of a vector $\vec{v}$. For some vector $\vec{v} \in \mathbb{R}^n$ of n dimensions,

$$||\vec{v}|| = \sqrt{\sum_{v_i \in \vec{v}} v_i^2} = \sqrt{v_1^2 + \dots + v_n^2} \quad (12)$$

3. The *direction of a vector* s.t. $\vec{v} \in \mathbb{R}^2$, is given by the angle that are made up by the v_1 and v_2 components; its trigonometric relations are given by:

$$(a) \tan \theta = \frac{v_2}{v_1}$$

$$(b) \theta = \arctan\left(\frac{v_2}{v_1}\right)$$

4. Given a k scalar, and some $\vec{u}$ and $\vec{v}$ vector, it is the case that,

$$(a) k\vec{v} = [kv_1, \dots, kv_n]$$

$$(b) \vec{u} + \vec{v} = [u_1 + v_1, \dots, u_n + v_n]$$

$$(c) \vec{u} - \vec{v} = [u_1 - v_1, \dots, u_n - v_n]$$

5. A *unit vector* is a special case of vector that has a magnitude of 1, i.e. $||\vec{v}|| = 1$. This unit vector can be worked out with the following formula:

$$\text{unit-vector}(\vec{v}) = \frac{1}{||\vec{v}||} \vec{v} \quad (13)$$

6. The *dot product* is defined as:

$$\vec{u} \cdot \vec{v} = \sum_{u_i \in \vec{u} \text{ and } v_i \in \vec{v}} u_i v_i = u_1 v_1 + \dots + u_n v_n \quad (14)$$

7. Given vectors $\vec{u}$ and $\vec{v}$ both $\in \mathbb{R}^2$ or $\in \mathbb{R}^3$, its dot product can also be expressed in terms of its magnitude and cosines; with their trigonometric relations being:

- $\vec{u} \cdot \vec{v} = ||\vec{u}|| ||\vec{v}|| \cos \theta$
- $\cos \theta = \frac{\vec{u} \cdot \vec{v}}{||\vec{u}|| ||\vec{v}||}$

8. Two vectors are *orthogonal* if the angle between the two vectors work out to 90° . Or, Orthogonal($\vec{u}, \vec{v}$) $\equiv [\vec{u} \cdot \vec{v} = 0]$
9. The determinant of a square matrix A , is denoted as:

$$\det A = \det(A) = \det[A]$$

In a case where A is a 2×2 matrix,

$$\det(A) = \det \left(\begin{bmatrix} a & b \\ c & d \end{bmatrix} \right) = ad - bc \quad (15)$$

In a case where A is a 3×3 matrix:

$$\det(A) = \det \left(\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \right) \quad (16)$$

The following table shows the arithmetic needed to be done to compute the determinant of a 3×3 matrix,

	Expression	Diagonal d_i
$\det(A) =$	$a_{11}a_{22}a_{33}$	$\longleftarrow (d_1)$
+	$a_{12}a_{23}a_{31}$	$\longleftarrow (d_2)$
+	$a_{13}a_{21}a_{32}$	$\longleftarrow (d_3)$
-	$a_{13}a_{22}a_{31}$	$\longleftarrow (d_4)$
-	$a_{11}a_{23}a_{32}$	$\longleftarrow (d_5)$
-	$a_{12}a_{21}a_{33}$	$\longleftarrow (d_6)$

$$\det(A) = \sum_{d_i \in \text{diagonals}} d_i$$

C MATLAB Solution Source Code

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Applied Linear Algebra (MAT-350): Project 2 Solution %
% By Alexander Ahmann <alexander.ahmann@snhu.edu>      %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%
% Part 1: Toy problems
%

%
% Q1: Consider the following 3 x 3 matrix (given in the problem
% set). Use the svd() function in MATLAB to compute A_1, the rank-1
% approximation of A. Clearly state what A_1 is, rounded to 4 deci-
% -mal places. Also, compute the root mean square error (RMSE)
% between A and A_1.
%

A = [1 2 3;
     3 3 4;
     5 6 7]

[U, S, V] = svd(A)

A_1 = U(:, 1:1) * S(1:1, 1:1) * V(:, 1:1)'
A_norm1 = norm(A - A_1, 'fro') / 3
A_rmse1 = sqrt(mean((A - A_1).^2, 'all'))
A_rank1 = rank(A_1)
% General format of "root mean square error" formula
%   'sqrt(mean((A - B).^2, 'all'))'
% where A and B are matrices of the same n x n.

%
% Q2: Use the svd() function in MATLAB to compute A_2, the rank-2
% approximation of A. Clearly state what A_2 is, rounded to 4
% decimal places. Also, compute the root mean square error (RMSE)
% between A and A_2. Which approximation is better, A_1 or A_2?
% Explain.
%
```

```

A_2 = A_1 + U(:, 1:2) * S(1:2, 1:2) * V(:, 1:2).';
A_norm2 = norm(A - A_2, 'fro') / 3
A_rmse2 = sqrt(mean((A - A_2).^2, 'all'))
A_rank2 = rank(A_2)

%
% Q3: For the given arbitrary 3 x 3 matrix A, the singular decom-
% -position is "A = USV" where "U = [u_1, u_2, u_3]". Use MATLAB to
% compute the dot product "d_1 = dot(u_1, u_2)". Also, use MATLAB
% to compute the cross product "c = cross(u_1, u_2)" and dot prod-
% -uct d_2 = dot(c, u_3). Clearly state the values for each of the
% -se computations. Do these values make sense? Explain.
%

u_1 = U(:,1)
u_2 = U(:,2)
u_3 = U(:,3)

d_1 = dot(u_1, u_2)
c = cross(u_1, u_2)
d_2 = dot(c, u_3)

%
% Q4: Using the matrix "U = [u_1, u_2, u_3]", determine whether or
% not the columns of U span  $\mathbb{R}^3$ . Explain your approach.
%

U_rref = rref(U) % returns three (3)
% pivots, therefore it spans.

%
% Part 2: Image Compression
%

clearvars

%
% Q5: Use the MATLAB "imshow()" function to load and display the
% image A stored in the provided MATLAB "image.mat" file. For
% the loaded image, derive the value of "k" that will result in
% a compression ratio of "CR  $\approx$  2". For this value of "k", con-

```

```

% -struct the rank-k approximation of the image.
%

load image.mat

imshow(A)
title("Given Image (No Compression)");

[m, n] = size(A) % NOTE: special thanks to K. B. Gardner for her advices
% given in "Project 2 Tips," which I references when coding the sol-
% -ution to this given question item.

image2matrix = im2double(double(A));
[U S V] = svd(image2matrix);

CR = 2
k_2 = round( (m * n) / (CR * (m + n + 1)))
% Derived by solving for "k" in:
% CR = (m * n) / (k * (m + n + 1))
Ak_CR2 = U(:, 1:k_2) * S(1:k_2, 1:k_2) * V(:, 1:k_2)';

Ak_CR2_rank = rank(Ak_CR2)
Ak_CR2 = uint8(round(Ak_CR2));

%
% Q6: Display the image and compute the root mean square error (RMSE)
% between the approximation and the original image. Make sure to
% include a copy of the approximate image in your report.
%

figure;
imshow(Ak_CR2);
title("Given Image (CR ~ 2)");

rmse_CR2 = sqrt(mean((A - Ak_CR2).^2, 'all'))

%
% Q7: Repeat steps and 5 and 6 for "CR ~ 10", "CR ~ 25", "CR ~ 75"
% Explain what trends you observe in the image approximation as CR the
% compression ratio increases and provide your recommendation for the
% best CR the compression ratio based on your observations. Make sure

```

```

%   to include a copy of the approximate images in your report.
%

%%
% Case CR ~ = 10
%%

CR = 10
k_10 = round( (m * n) / (CR * (m + n + 1)))
% Derived by solving for "k" in:
% CR = (m * n) / (k * (m + n + 1))
Ak_CR10 = U(:, 1:k_10) * S(1:k_10, 1:k_10) * V(:, 1:k_10)';

Ak_CR10_rank = rank(Ak_CR10)
Ak_CR10 = uint8(round(Ak_CR10));

figure;
imshow(Ak_CR10);
title("Given Image (CR ~ = 10)");

rmse_CR10 = sqrt(mean((A - Ak_CR10).^2, 'all'))

%%
% Case CR ~ = 25
%%

CR = 25
k_25 = round( (m * n) / (CR * (m + n + 1)))
% Derived by solving for "k" in:
% CR = (m * n) / (k * (m + n + 1))
Ak_CR25 = U(:, 1:k_25) * S(1:k_25, 1:k_25) * V(:, 1:k_25)';

Ak_CR25_rank = rank(Ak_CR25)
Ak_CR25 = uint8(round(Ak_CR25));

figure;
imshow(Ak_CR25);
title("Given Image (CR ~ = 25)");

rmse_CR25 = sqrt(mean((A - Ak_CR25).^2, 'all'))

```

```

%%
% Case CR ~ = 75
%%

CR = 75
k_75 = round( (m * n) / (CR * (m + n + 1)))
    % Derived by solving for "k" in:
    % CR = (m * n) / (k * (m + n + 1))
Ak_CR75 = U(:, 1:k_75) * S(1:k_75, 1:k_75) * V(:, 1:k_75)';

Ak_CR75_rank = rank(Ak_CR75)
Ak_CR75 = uint8(round(Ak_CR75));

figure;
imshow(Ak_CR75);
title("Given Image (CR ~ = 75)");

rmse_CR75 = sqrt(mean((A - Ak_CR75).^2, 'all'))

%%%%%%%%%%
% Done %
%%%%%%%%%%

clearvars

```

D Results of Running the MATLAB solution

```
>> solution
```

```
A =
```

1	2	3
3	3	4
5	6	7

```
U =
```

-0.2904	0.9504	-0.1114
-0.4644	-0.2418	-0.8520

-0.8367 -0.1957 0.5115

S =

12.5318	0	0
0	0.9122	0
0	0	0.3499

V =

-0.4682	-0.8261	-0.3136
-0.5581	0.0012	0.8298
-0.6851	0.5635	-0.4616

A_1 =

1.7039	2.0313	2.4935
2.7243	3.2477	3.9867
4.9087	5.8517	7.1832

A_norm1 =

0.3257

A_rmse1 =

0.3257

A_rank1 =

1

A_2 =

2.6917	4.0636	5.4755
5.6308	6.4950	7.8490
9.9648	11.7032	14.2659

A_norm2 =

4.1789

A_rmse2 =

4.1789

A_rank2 =

2

u_1 =

-0.2904
-0.4644
-0.8367

u_2 =

0.9504
-0.2418
-0.1957

u_3 =

-0.1114
-0.8520
0.5115

d_1 =

1.6653e-16

c =

-0.1114

-0.8520

0.5115

d_2 =

1.0000

U_rref =

1	0	0
---	---	---

0	1	0
---	---	---

0	0	1
---	---	---

m =

2583

n =

4220

CR =

2

k_2 =

801

Ak_CR2_rank =

801

rmse_CR2 =

2.2389

CR =

10

k_10 =

160

Ak_CR10_rank =

160

rmse_CR10 =

5.1769

CR =

25

k_25 =

64

```
Ak_CR25_rank =
```

```
64
```

```
rmse_CR25 =
```

```
6.3933
```

```
CR =
```

```
75
```

```
k_75 =
```

```
21
```

```
Ak_CR75_rank =
```

```
21
```

```
rmse_CR75 =
```

```
7.6909
```

```
>>
```